ESKOOLIA SCHOOL ERP

Role & Permission Module

User Acceptance Testing (UAT) Report

With AI Integration Recommendations & Improvement Roadmap

Date: March 28, 2026
Environment: http://187.127.137.154/
Tested By: Automated UAT Agent

Verdict: 10 CRITICAL | 8 FAIL | 4 WARNINGS | 28 PASS (50 total)

1. Executive Summary

This report presents the findings of a comprehensive end-to-end User Acceptance Test (UAT) of the Role & Permission module in Eskoolia School ERP. Testing covered 50 test cases across 4 categories: Role CRUD (15), Permission Assignment (8), Access Control Enforcement (14), and UI/UX (13).

The module is in a partially functional state. Basic CRUD operations on roles work. However, a CRITICAL security vulnerability was discovered: permissions are enforced ONLY on the access-control API endpoints themselves. All other module APIs (students, fees, HR, finance, exams, library, inventory, behaviour) have ZERO permission enforcement. A Teacher user with 0 assigned permissions can read, modify, and delete data across every module.

This means the permission system is purely cosmetic today — it controls UI sidebar visibility but does not enforce actual data access at the API layer.

Test Results Summary

Category	PASS	FAIL	CRITICAL	WARN
Role CRUD Operations	11	2	0	2
Permission Assignment	6	1	1	0
Access Control Enforcement	5	0	9	0
UI/UX Quality	6	5	0	2
TOTAL	28	8	10	4

2. Module Architecture

2.1 UI Pages

The Role Permission module has 4 frontend pages accessible from the sidebar navigation:

1. /roles — Role list with Add/Edit/Delete and Assign Permission buttons
2. /roles/assign-permission — Module-based checkbox grid for assigning permissions to a role
3. /roles/login-permission — Enable/disable login access per user, with password reset
4. /roles/due-fees-login-permission — Block student/parent login based on unpaid fees

2.2 API Endpoints

Backend runs on Django REST Framework. All endpoints are under /api/v1/access-control/:

Method	Endpoint	Purpose
GET/POST	/roles/	List and create roles
GET/PUT/PATCH/DELETE	/roles/:id/	Single role CRUD
GET	/permissions/	List all 96 permissions
GET	/roles/permission-tree/?role=X	Module-grouped permission tree
GET/POST/DELETE	/user-roles/	User-to-role assignments
GET	/login-access-control/	Roles + classes for login control
GET	/login-access-control/users/?role=X	Users by role with login status
POST	/login-access-control/toggle/	Enable/disable user login
POST	/login-access-control/reset-password/	Reset user password to default
GET	/due-fees-login-permission/	Classes/sections for fee-based login control

2.3 Data Model

Roles: 5 existing (Admin, Driver, Student, Super Admin, Teacher). Each role has an array of permission_ids. Permissions: 96 total across 13 modules. Permission code format: module.entity.verb (e.g., academics.add_homework.view). Available verbs: view, read, manage, add, edit, delete. User-Role assignments: 3 existing mappings (2 Teachers, 1 Driver).

3. Detailed Test Results

3.1 Role CRUD Operations

ID	Test Case	Result	Status
TC-R01	Create role via API (POST)	201 Created, role returned with ID	PASS
TC-R02	Read role list (GET)	200 OK, all 5 roles returned	PASS
TC-R03	Read single role (GET /:id)	200 OK, full role object returned	PASS
TC-R04	Update role (PATCH)	200 OK, name updated correctly	PASS
TC-R05	Update role (PUT)	200 OK, full replace successful	PASS
TC-R06	Delete role (DELETE)	204 No Content, role removed	PASS
TC-R07	Delete non-existent role	Returns 500 instead of 404	FAIL
TC-R08	Create duplicate role name	Returns 500 instead of 400 validation error	FAIL
TC-R09	Create with empty name	400 with validation message	PASS
TC-R10	Create with no name field	400 with required field message	PASS
TC-R11	Create with 500-char name	400 max length validation	PASS
TC-R12	Create with whitespace-only name	400 rejected correctly	PASS
TC-R13	Create with special characters	201 accepted (debatable)	WARNING
TC-R14	Create with Unicode (Telugu)	201 accepted, stored correctly	PASS
TC-R15	Search filter on roles list	Returns all 5 roles (no server-side filter)	WARNING

3.2 Permission Assignment

ID	Test Case	Result	Status
TC-P01	Assign permissions to role (PATCH permission_ids)	200 OK, permissions saved	PASS
TC-P02	Remove all permissions (empty array)	200 OK, permissions cleared	PASS
TC-P03	Assign invalid permission IDs	400 validation error returned	PASS
TC-P04	Assign mix of valid+invalid IDs	400 rejects entire batch (atomic)	PASS
TC-P05	Assign duplicate permission IDs	200 OK, duplicates deduplicated	PASS
TC-P06	Permission tree endpoint	200 OK, 13 modules with selected flags	PASS
TC-P07	Permission tree - module name field	Module name is undefined in response	FAIL
TC-P08	Permissions only have view verb	Only 'view' verb for most modules; no create/edit/delete granularity	CRITICAL

3.3 Access Control Enforcement

These tests logged in as a Teacher user (0 permissions assigned) and attempted to access various module APIs.

ID	Test Case	Result	Status
TC-A01	Unauthenticated access to roles API	401 Unauthorized	PASS
TC-A02	Invalid token access	401 Unauthorized	PASS
TC-A03	Teacher blocked from roles API	403 Forbidden	PASS
TC-A04	Teacher blocked from permissions API	403 Forbidden	PASS
TC-A05	Teacher blocked from modifying Admin role	403 Forbidden	PASS
TC-A06	Teacher can read ALL student data	200 OK (should be 403)	CRITICAL
TC-A07	Teacher can read ALL finance data	200 OK (should be 403)	CRITICAL
TC-A08	Teacher can read ALL HR/payroll data	200 OK (should be 403)	CRITICAL
TC-A09	Teacher can DELETE students	204 No Content (should be 403)	CRITICAL
TC-A10	Teacher can read fee payments	200 OK (should be 403)	CRITICAL
TC-A11	Teacher can read exam data	200 OK (should be 403)	CRITICAL
TC-A12	Teacher can read library data	200 OK (should be 403)	CRITICAL
TC-A13	Teacher can read inventory data	200 OK (should be 403)	CRITICAL
TC-A14	Teacher can read behaviour records	200 OK (should be 403)	CRITICAL

3.4 UI/UX Testing

ID	Test Case	Result	Status
TC-U01	Add Role form - empty submit	No validation message shown; silent failure	FAIL
TC-U02	Edit Role - pre-fills current name	Works correctly with Save/Cancel buttons	PASS
TC-U03	Search box on role list	Client-side filter works correctly	PASS
TC-U04	Assign Permission page - module cards	Expand/collapse works, checkboxes functional	PASS
TC-U05	Assign Permission - URL parameter ignored	URL /assign-permission/2 loads Admin, not Teacher	FAIL
TC-U06	Assign Permission - role dropdown switch	Correctly loads permission state for selected role	PASS
TC-U07	Login Permission - role search	Shows users with login toggle, password controls	PASS
TC-U08	Login Permission - Admin role excluded	Admin not in dropdown (correct security practice)	PASS
TC-U09	Due Fees - section dropdown not filtered	Shows ALL 36 sections regardless of class selected	FAIL
TC-U10	Due Fees - search returns no results	Search for Class 1 returns empty; no API call made	FAIL
TC-U11	Delete role - no confirmation dialog	No prompt before destructive action	FAIL
TC-U12	Breadcrumb navigation	Present but inconsistent across sub-pages	WARNING
TC-U13	Role type column shows 'Custom' for all	No 'System' type; is_system field unused	WARNING

4. Critical Issues & Bugs

4.1 [CRITICAL] No Backend Permission Enforcement on Module APIs

Severity: CRITICAL | Priority: P0 | Affects: ALL modules

The permission system only gates the access-control endpoints (roles, permissions). Every other API endpoint (students, fees, HR, payroll, finance, exams, library, inventory, behaviour, admissions) performs NO permission check. A user with the Teacher role and ZERO permissions assigned can: read all student records, read all financial data, read all HR/payroll records, and even DELETE student records. This makes the entire permission system cosmetic — it hides sidebar items in the UI but does not protect data at the API layer.

Recommendation: Implement a DRF permission class (e.g., HasModulePermission) that checks the user's role permissions on every API view. Apply it globally via DEFAULT_PERMISSION_CLASSES in settings.py.

4.2 [CRITICAL] Permission Granularity is View-Only

Severity: CRITICAL | Priority: P1

Of 96 permissions, most only have a 'view' verb. Only the admin_section module has add/edit/delete granularity. This means even if backend enforcement existed, you cannot grant a Teacher 'view students' without also implicitly granting 'delete students'. The permission model needs CRUD-level granularity for every module entity.

4.3 [FAIL] Duplicate Role Name Returns 500

Severity: HIGH | Priority: P2

Creating a role with a name that already exists returns HTTP 500 (internal server error) instead of 400 with a user-friendly validation message. The database unique constraint fires but is not caught by the API serializer.

4.4 [FAIL] GET/DELETE Non-existent Role Returns 500

Severity: MEDIUM | Priority: P2

Requesting GET or DELETE on a role ID that does not exist returns HTTP 500 instead of 404 Not Found. The view is not handling DoesNotExist exceptions.

4.5 [FAIL] Assign Permission URL Parameter Ignored

Severity: MEDIUM | Priority: P3

Navigating to /roles/assign-permission/2 (Teacher) always loads Admin (id=1) permissions. The URL path parameter is ignored; the frontend always defaults to the first role in the dropdown.

4.6 [FAIL] Due Fees Login Permission Page Non-Functional

Severity: HIGH | Priority: P2

The Due Fees Login Permission page has two bugs: (1) The Section dropdown shows all 36 sections from all classes instead of filtering by selected class. (2) Clicking Search with a class selected makes no API call and returns empty results. The search functionality appears to be unimplemented or broken.

4.7 [FAIL] No Confirmation Dialog on Role Delete

Severity: MEDIUM | Priority: P3

Clicking the Delete button on a role performs the deletion immediately without any confirmation prompt. This is a destructive action that should require user confirmation.

4.8 [FAIL] Empty Form Submit Shows No Validation

Severity: LOW | Priority: P4

Submitting the Add Role form with an empty name field silently fails with no visual feedback. Should show inline validation or a toast message.

5. Improvement Recommendations

5.1 Backend Permission Enforcement (P0)

Create a custom DRF permission class that maps each API viewset to its required permission codes. Example: StudentsViewSet requires student_info.student_list.view for GET, student_info.add_student.add for POST, etc. Register it in DEFAULT_PERMISSION_CLASSES so it applies globally. This single change would close the biggest security gap.

5.2 Expand Permission Granularity (P1)

Current state: 96 permissions, mostly view-only. Target state: ~300+ permissions with full CRUD verbs (view, add, edit, delete) for every module entity. Generate these programmatically from the model registry. Example: for the Students module, create: students.student_list.view, students.student_list.add, students.student_list.edit, students.student_list.delete, students.student_category.view, students.student_category.add, etc.

5.3 Role Templates & Hierarchy

Add pre-built role templates (Principal, Vice Principal, Class Teacher, Subject Teacher, Accountant, Librarian, Transport Manager, Parent, Student) with sensible default permissions. Implement role hierarchy so a Principal inherits all Teacher permissions automatically.

5.4 Audit Logging

Every permission change, role assignment, and login permission toggle should be logged with timestamp, actor, action, and before/after values. This is essential for compliance in educational institutions.

5.5 API Error Handling

Replace all 500 errors with proper 400/404/409 responses. Use DRF exception handlers to catch IntegrityError (duplicate names), DoesNotExist (invalid IDs), and return structured error JSON with user-readable messages.

5.6 UI Improvements

Add confirmation dialogs for all destructive actions (delete role, remove permissions). Show inline validation on form fields. Fix the section dropdown cascading filter on Due Fees page. Show toast notifications on success/failure. Add bulk permission assignment (e.g., 'Grant all view permissions').

6. AI Integration Recommendations

If Claude were building this system to production-grade standards, these AI-powered features would set it apart from every competing school ERP:

6.1 AI-Powered Role Recommendation Engine

When creating a new staff member and assigning them a role, the system should use AI to analyze their job title, department, and responsibilities, then automatically recommend a role with pre-selected permissions. For example, entering 'Chemistry Lab Assistant' would suggest the Teacher role with academics, examination (view-only), and student_info (view-only) permissions, but NOT fees, HR, or finance access. The AI considers the principle of least privilege.

6.2 Anomaly Detection on Permission Usage

Train a model on normal access patterns per role. Flag anomalies in real-time: a Teacher accessing finance APIs at 2 AM, a Driver downloading the full student list, or a Student role making 500 API calls in an hour. Display alerts on the admin dashboard and optionally auto-revoke sessions. This catches both insider threats and compromised accounts.

6.3 Natural Language Permission Queries

Allow admins to ask questions like 'Who can delete student records?' or 'What can the Driver role access?' and get instant, accurate answers by querying the permission graph. Extend this to 'Show me everyone who accessed fee data in the last week' for audit purposes. Implementation: an LLM tool-use agent backed by the permission and audit log APIs.

6.4 Smart Permission Conflict Detection

When an admin assigns permissions, AI should flag conflicts and risks: 'Granting fee_payment.add to the Student role allows students to create fake payment records. Are you sure?' or 'The Teacher role has delete_student but no view_student — this combination is likely a mistake.' This prevents misconfiguration at the point of entry.

6.5 Automated Compliance Reporting

Generate periodic reports: 'These 3 staff members have permissions they have never used in 90 days — consider revoking.' 'The Accountant role was granted HR access on Jan 15 but this was never approved in the change log.' 'All parent accounts have correct fee-based login restrictions.' This replaces manual quarterly audits.

6.6 AI-Driven Onboarding Wizard

When a new school starts using the ERP, an AI wizard interviews the principal about their organizational structure: How many departments? Do class teachers handle attendance? Does the accountant also manage inventory? Based on answers, it auto-generates the complete role and permission structure, saving days of manual setup.

7. Conclusion & Verdict

The Role & Permission module has solid foundational architecture: clean API design, JWT authentication, module-based permission grouping, and a functional UI for basic role management. However, it has a showstopper security flaw: permissions are not enforced at the API layer beyond the access-control endpoints themselves.

Until backend permission enforcement is implemented across ALL module APIs, the system CANNOT be used in production. Any authenticated user can access and modify any data in any module, regardless of their assigned role and permissions.

The recommended priority order is: (1) Backend permission enforcement on all APIs. (2) Expand permission granularity to CRUD per entity. (3) Fix API error handling (500s to 400/404). (4) Fix UI bugs (Due Fees page, URL params, validation). (5) Add audit logging. (6) Implement AI-powered features for competitive differentiation.

With these improvements, the module would meet enterprise-grade standards suitable for school ERP deployment. The AI integration recommendations would position Eskoolia as the most intelligent school management platform on the market.